

# Chapter 1

## DBMS Internals and the Life of a Query

What a database system is made of, and what happens between  
typing a query and seeing the answer

Advanced Data Management

Based on Chapter 1 slides

### How to read this chapter

This chapter is the map for the whole course. Every later topic (indexing, joins, optimisation, transactions, recovery, NoSQL) is one box in the picture we draw here. So the goal is not to master any one mechanism yet, but to know *where each mechanism lives and why it has to exist*. Whenever a later chapter opens a box, come back to this one and find the box. The “Pause and check” boxes are meant to be answered before reading on.

## 1 Why We Need a DBMS

### 1.1 Motivation: the school that stored everything in files

Suppose a school keeps its data in plain files: `students.csv`, `grades.csv`, `courses.csv`. The first week it works fine. Then the problems arrive, one by one, and each of them is a reason a database management system exists.

- **Querying is manual.** “What is the average grade per programme?” means writing a new program that opens two files, matches rows by student id, groups, and averages. The next question means another program.
- **No concurrency.** Two secretaries update `grades.csv` at the same moment. Each reads the file, changes one line, writes the whole file back. The second write silently erases the first.
- **No crash safety.** The power fails halfway through writing the file. What is on disk is half old, half new, and nobody can tell which half is which.
- **No integrity.** Nothing stops a grade of 500, a student with no name, or a grade for a student id that does not exist.
- **Redundancy and inconsistency.** The student’s name is copied into every file that mentions them. Change it in one place and the copies drift apart.

- **No access control.** Anyone who can open the file sees every column, salaries included.
- **Poor performance at scale.** Finding one student means reading the whole file, every time.

The analogy I like: a filing cabinet versus a library. A filing cabinet stores paper. A library stores paper *and* provides a catalogue, a librarian who finds things for you, rules about who may borrow what, and a system that survives a book being mis-shelved. A DBMS is the library, not the cabinet.

**History.** Before relational systems (Codd, 1970), “databases” were exactly this: application programs with hand-coded file access. IBM’s System R (1974–79) and Berkeley’s Ingres were the first to prove a general-purpose engine could beat hand-written code.

### Concept: What a DBMS provides, as services

A database management system turns each of those problems into a service the application no longer has to implement:

| Service                   | What it means                                                                               |
|---------------------------|---------------------------------------------------------------------------------------------|
| Declarative querying      | You say <i>what</i> you want (SQL); the system works out <i>how</i> .                       |
| Concurrency control       | Many users at once, and the result is still as if they ran one at a time.                   |
| Recovery                  | After a crash, everything that was committed is still there, and nothing half-done remains. |
| Efficient access          | Indexes, buffering and query optimisation find rows without scanning everything.            |
| Integrity and constraints | Keys, types, and rules the data must obey, enforced by the system.                          |
| Security                  | Who may see or change what, per table and per column.                                       |
| Data independence         | Change the physical storage (add an index, move to SSD) without rewriting any application.  |

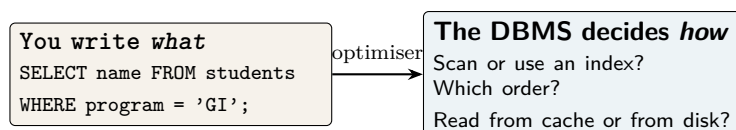
## 1.2 The key idea: declarative versus procedural

Of all the services above, one shapes the entire course. When you write

```
SELECT name FROM students WHERE program = 'GI';
```

you have described a *result*: the names of the students in programme GI. You have said nothing about how to get it. Should the system read the whole table and test each row? Is there an index on **program** that jumps straight to the GI rows? Is the table already in memory, or on disk? Which order should things happen in? Every one of those is a decision, and the DBMS makes it, not you.

**Key insight.** SQL is a description of the answer. The plan is the recipe. The gap between the two is what this course studies.



Turning “what” into an efficient, correct, crash-safe “how” is exactly what

the internals we study this semester do. Keep this sentence in mind: every mechanism in the course is in service of closing that gap.

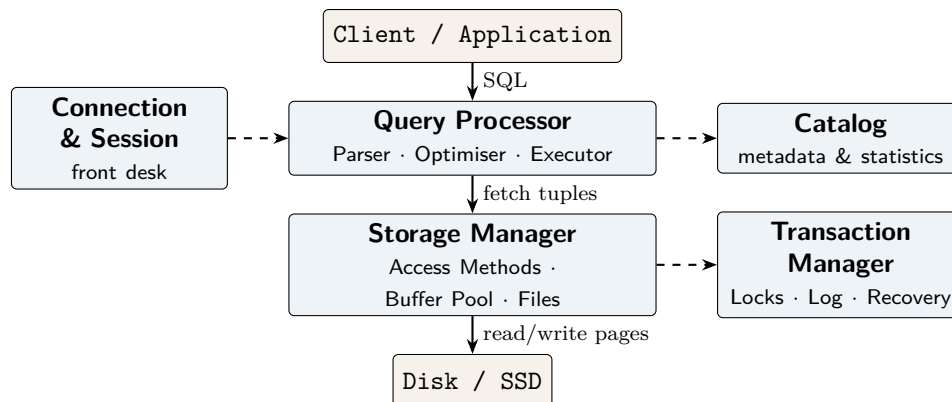
### Summary

Flat files fail on querying, concurrency, crashes, integrity, redundancy, security and scale. A DBMS provides each of these as a service. The most important is declarative querying: the user states the result, the system chooses the strategy, and that freedom to choose is what makes everything else possible.

## 2 Inside a DBMS: The Five Subsystems

### 2.1 Conceptual overview

A database engine is not one program but several cooperating parts. The standard reference for this structure is Hellerstein, Stonebraker and Hamilton, *Architecture of a Database System* (2007). There are five subsystems, and it helps to see them on one picture before meeting each one.



Read it top to bottom. A client sends SQL. The query processor turns it into a plan and runs it, asking the storage manager for tuples. The storage manager reads and writes pages on disk. On the side, the catalog tells the query processor what tables exist, and the transaction manager makes sure concurrent clients and crashes cannot corrupt anything.

### 2.2 Subsystem 1: connection and session manager

#### Mechanism: The front desk

- Listens for client connections and authenticates them.
- Gives each session its own context: current database, settings, temporary tables.
- Manages a pool of worker processes or threads so many clients run at once.

- Enforces limits (maximum connections, timeouts) so one client cannot starve the rest.

Think of it as the front desk of a building: it decides who gets in and hands each visitor a private workspace. It does no database work itself, but without it there is no “many users at once”.

#### In the real world.

PostgreSQL forks one operating-system process per connection. That is why connection pools such as PgBouncer exist: opening a connection is expensive, so applications reuse them.

## 2.3 Subsystem 2: query processor

### Mechanism: From SQL string to answer, in three stages

- **Parser:** checks the SQL grammar and builds a syntax tree.
- **Optimiser:** rewrites the query and picks the cheapest execution plan.
- **Executor:** runs that plan, pulling and combining tuples.

This is the subsystem that closes the “what to how” gap. Section 3 follows a query through it in detail. Operator algorithms (Chapter 4) and query optimisation (Chapter 5) live entirely inside it.

## 2.4 Subsystem 3: storage manager

### Mechanism: Everything about bytes on disk and in memory

- **Files and pages:** how tables and rows are laid out on disk (Chapter 2).
- **Access methods:** indexes such as B+ trees and hash indexes (Chapter 3).
- **Buffer pool:** the in-memory cache of disk pages (Chapter 6).

Why does it get three chapters? Because disk and SSD are thousands of times slower than memory, so *how and when pages are read* dominates performance. Most database tuning happens here.

## 2.5 Subsystem 4: transaction manager

### Mechanism: Correct under concurrency, safe under crashes

- **Lock manager:** coordinates concurrent transactions (Chapters 7–8).
- **Log manager:** records every change so it can be replayed or undone.
- **Recovery:** restores a consistent state after a failure (Chapter 9).

The promise it makes is simple to state and hard to keep: *either a transaction happens completely, or not at all*, even if the power fails halfway. This is the “ACID” guarantee, defined precisely in Chapter 7. The bank transfer is the classic picture: debit one account, credit another. A crash between the two must not leave money vanished.

## 2.6 Subsystem 5: the catalog

### Mechanism: The database's description of itself

- Which tables and columns exist, with their types and constraints.
- Which indexes exist, and on which columns.
- Statistics: row counts, value distributions. The optimiser lives on these.

The elegant part: the catalog is just data, stored in tables like everything else. In PostgreSQL you can query it:

```
SELECT * FROM pg_stats WHERE tablename = 'customers';
SELECT * FROM information_schema.columns WHERE table_name = 'orders';
```

## 2.7 How the subsystems map to this course

| Subsystem / theme                                         | Chapters   |
|-----------------------------------------------------------|------------|
| Storage manager (files, indexes, buffer)                  | 2, 3, 6    |
| Query processor (operators, optimisation)                 | 4, 5       |
| Transaction manager (transactions, concurrency, recovery) | 7, 8, 9    |
| Scaling out (parallel and distributed)                    | 10         |
| Modern systems (NoSQL, lakehouse, vectors)                | 11, 12, 13 |

Each chapter zooms into one row of this table.

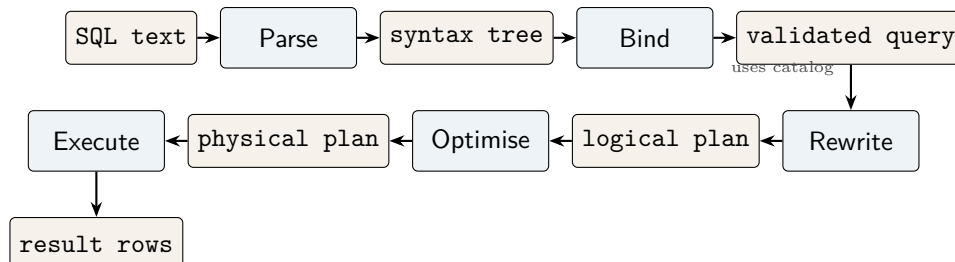
### Summary

Five subsystems: the connection manager admits clients; the query processor turns SQL into a plan and runs it; the storage manager owns pages, files, indexes and the buffer pool; the transaction manager provides locking, logging and recovery; the catalog describes everything and is itself stored as tables.

## 3 The Life of a Query

### 3.1 Conceptual overview: five stages

One SQL string goes in, rows come out. In between, five stages, each producing a more concrete object than the one before.



We will walk every box with one running example, and I will show you the actual object each stage produces, so “syntax tree” and “logical plan” stop being words and become things you can draw.

## 3.2 The running example

```
SELECT c.name, SUM(o.amount) AS total
FROM   customers c
JOIN   orders  o ON o.cust_id = c.id
WHERE  c.country = 'MA'
GROUP BY c.name;
```

In plain words: for every Moroccan customer, add up the value of their orders. To make the stages concrete, here is a tiny instance of the two tables that we will carry all the way to the result.

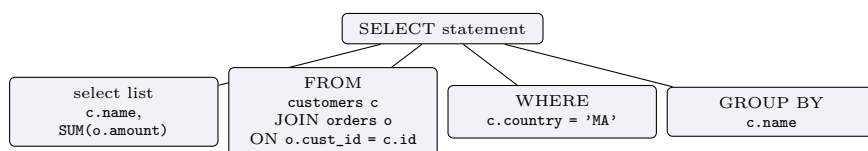
| customers |      |         | orders |         |        |
|-----------|------|---------|--------|---------|--------|
| id        | name | country | id     | cust_id | amount |
| 1         | Sara | MA      | 10     | 1       | 40     |
| 2         | Amir | FR      | 11     | 3       | 15     |
| 3         | Lina | MA      | 12     | 1       | 90     |
|           |      |         | 13     | 2       | 70     |

Expected answer: Sara 130, Lina 15. Amir is French and is excluded.

## 3.3 Stage 1: parsing

### Mechanism: Parser

**Input:** the SQL text. **Output:** a syntax tree mirroring the query’s structure. A lexer first splits the text into tokens (**SELECT**, **c.name**, **,**, **...**); a parser then checks the tokens against the SQL grammar and builds the tree.



Two things to understand about this stage. First, it is **pure syntax**: a missing comma or a misspelled keyword is rejected here, with a message like “syntax error at or near **FORM**”. Second, and this surprises people, **the parser has no idea whether customers or country exist**. To the parser, **customers** is just an identifier in the right grammatical position. Whether there is such a table is a question for the next stage. If you misspell a column name, this stage is perfectly happy.

### 3.4 Stage 2: binding and semantic analysis

#### Mechanism: Binder

**Input:** the syntax tree. **Output:** a validated query in which every name is resolved to a catalog object and every expression has a known type. This is where the query meets the catalog.

Concretely, the binder does four kinds of checks on our query:

1. **Existence.** Do tables `customers` and `orders` exist? Do columns `name`, `amount`, `country`, `cust_id`, `id` exist on the tables they are used with?
2. **Resolution.** The aliases `c` and `o` are bound to their tables. Every column reference becomes a pointer to a specific catalog entry, with its type attached: `c.country` is `text`, `o.amount` is `numeric`.
3. **Type checking.** Is `SUM(o.amount)` valid? Yes, `SUM` accepts `numeric`. Is `c.country = 'MA'` comparing compatible types? Yes, `text` with `text`. Is `c.name` allowed in the select list? Yes, because it is in `GROUP BY`.
4. **Permissions.** May this user read both tables?

The output is the same tree, but now every leaf carries a resolved reference and a type, much like the type-decorated AST in a compiler. Errors such as “column `amuont` does not exist” are raised here.

**Key insight.** “Column does not exist” is a *binding* error, not a parse error. The parser cannot know; only the catalog can.

### 3.5 Stage 3: rewriting into a logical plan

#### Mechanism: Rewriter

**Input:** the validated query. **Output:** a tree of relational-algebra operators, the **logical plan**. It says *what* to compute, still not *how*.

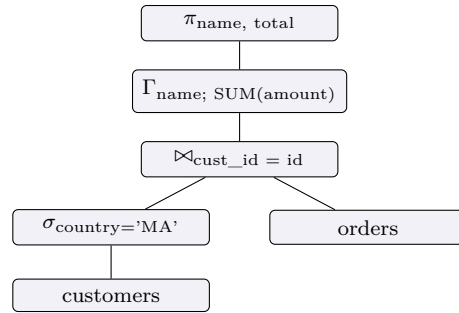
Relational algebra is the small set of operations every SQL query reduces to. The four we need:

| Symbol         | Name       | Meaning, and the SQL clause it comes from                           |
|----------------|------------|---------------------------------------------------------------------|
| $\sigma_p$     | selection  | keep the rows satisfying predicate $p$ ( <b>WHERE</b> )             |
| $\pi_{a,b}$    | projection | keep only columns $a, b$ (the <b>SELECT</b> list)                   |
| $\bowtie_p$    | join       | pair up rows from two inputs where $p$ holds ( <b>JOIN ... ON</b> ) |
| $\Gamma_{g,f}$ | grouping   | group by $g$ , compute aggregate $f$ per group ( <b>GROUP BY</b> )  |

For our query the logical plan is, roughly,

$$\pi_{\text{name, total}} \left( \Gamma_{\text{name}; \text{SUM}(\text{amount})} (\sigma_{\text{country}='MA'} (\text{customers}) \bowtie_{\text{cust\_id=id}} \text{orders}) \right)$$

Read it inside out: filter customers to Morocco, join with orders, group by name summing amount, project the two output columns. As a tree:



Rule-based rewrites tidy the tree: views are expanded into their definitions, subqueries are flattened into joins where possible, predicates are simplified ( $x = 1 \text{ AND } x = 1$  becomes  $x = 1$ ). Notice one rewrite already happened above: the filter on `country` was pushed *below* the join, next to `customers`. This is always safe (the filter only mentions customer columns) and always good (fewer rows enter the join). It is the single most important rewrite, and you will meet it again in Chapter 5 under the name *predicate pushdown*.

### 3.6 Stage 4: optimisation into a physical plan

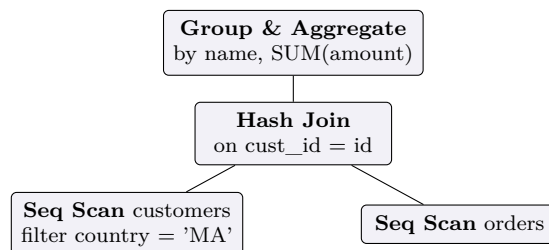
#### Mechanism: Optimiser

**Input:** the logical plan. **Output:** a **physical plan**: the same tree, but with every operator replaced by a concrete algorithm and every table access replaced by a concrete access path.

The logical plan left three kinds of decision open, and the optimiser must make all of them:

- **Which access path?** Read all of `customers` sequentially and test each row for `country = 'MA'`, or use an index on `country` to jump to the Moroccan rows only?
- **Which join algorithm?** Nested loop, sort-merge, or hash join (Chapter 4). Each is best in a different situation.
- **Which order?** Filter Moroccan customers first and then join (shrinks the work), or join everything and filter afterwards?

It estimates the cost of each option using catalog statistics (how many rows, how many are Moroccan, is there an index) and keeps the cheapest. One possible outcome:



Read it bottom-up: scan, join, then group. This tree is what the executor

runs. Compare it with the logical plan: same shape, but “join” has become “hash join”, “customers” has become “sequential scan of customers”, and the projection has been folded into the aggregate. Many correct plans exist for one query; the optimiser’s job is to pick a fast one. That is the heart of Chapter 5.

### 3.7 Stage 5: execution, the iterator model

#### Mechanism: Executor and the Volcano / iterator model

Each operator in the physical plan is a small object with three methods:

- `open()`: get ready (allocate a hash table, open a file, call `open()` on children).
- `next()`: produce the *next* output tuple, asking children for their tuples as needed. Returns nothing when exhausted.
- `close()`: clean up.

The top operator is asked for tuples. The request flows *down* the tree; tuples flow *up*.

**In the real world.** In PostgreSQL, prefix any query with `EXPLAIN` to see the physical plan the optimiser chose, and `EXPLAIN ANALYZE` to run it and see actual row counts and timings next to the estimates. You will use this in every lab.

Why this design? Because it makes every operator interchangeable. A hash join does not care whether its child is a sequential scan, an index scan, or another join; it just calls `next()`. And tuples are **pipelined** one at a time: the join does not wait for the whole scan to finish before starting, and usually nothing has to be materialised into a temporary table. Leaf operators (the scans) are the only ones that talk to the storage manager, asking for pages from the buffer pool or from disk.

Here is the skeleton of two operators, to make the pattern concrete.

```
class SeqScan:                                # leaf: reads pages from the storage manager
    def __init__(self, table, pred=None): self.table, self.pred = table, pred
    def open(self): self.rows = iter(self.table.all_rows()) # page by page
    def next(self):
        for r in self.rows:
            if self.pred is None or self.pred(r): return r
        return None
    def close(self): pass

class HashJoin:                                # build a hash table on the left, probe with the
    right
    def __init__(self, left, right, lkey, rkey):
        self.left, self.right, self.lkey, self.rkey = left, right, lkey, rkey
    def open(self):
        self.left.open(); self.right.open(); self.ht = {}
        while (r := self.left.next()) is not None: # BUILD: drain left
            self.ht.setdefault(self.lkey(r), []).append(r)
        self.pending = []
    def next(self): # PROBE: one right
        tuple at a time
        while not self.pending:
            r = self.right.next()
            if r is None: return None
            self.pending = [l | r for l in self.ht.get(self.rkey(r), [])]
        return self.pending.pop()
    def close(self): self.left.close(); self.right.close()
```

### Worked example: Tracing execution on the tiny instance

Let us run the physical plan on our three customers and four orders and watch tuples move.

1. **open() cascades down.** Aggregate opens the hash join. The hash join opens both scans, then immediately drains its *left* child to build a hash table: it calls `next()` on the customers scan repeatedly. The scan reads the customers page, tests each row against `country = 'MA'`, and returns Sara(1) and Lina(3); Amir is filtered out and never leaves the scan. Hash table:  $\{1 \mapsto \text{Sara}, 3 \mapsto \text{Lina}\}$ .
2. **Aggregate calls next() on the join.** The join calls `next()` on the orders scan, gets order 10 (cust 1, 40), probes the hash table with key 1, finds Sara, and returns the joined tuple (Sara, 40).
3. **Aggregate keeps pulling.** Next: order 11 (cust 3, 15) probes key 3, finds Lina, returns (Lina, 15). Next: order 12 (cust 1, 90) returns (Sara, 90). Next: order 13 (cust 2, 70) probes key 2, finds nothing, so the join pulls again, the scan returns `None`, and the join returns `None`.
4. **Aggregate finishes.** It has been accumulating in its own hash table:  $\{\text{Sara} \mapsto 130, \text{Lina} \mapsto 15\}$ . Now that its child is exhausted, it emits (Sara, 130) and (Lina, 15), one per `next()` call. Then `close()` cascades down.

Two observations. Order 13 was read from disk and then discarded; the optimiser accepted that cost because it judged that filtering customers first was cheaper than filtering orders. And the aggregate is a *blocking* operator: it cannot emit anything until it has seen every input, unlike the join, which streamed. Chapter 4 classifies operators this way.

## 3.8 The whole journey at a glance

| Stage    | Produces                        | Studied in    |
|----------|---------------------------------|---------------|
| Parse    | syntax tree                     | this chapter  |
| Bind     | validated query (uses catalog)  | this chapter  |
| Rewrite  | logical plan (algebra)          | Chapter 5     |
| Optimise | physical plan (uses statistics) | Chapter 5     |
| Execute  | result rows (operators)         | Chapters 4, 6 |

### Trade-offs: Why keep the logical plan separate from the physical plan?

Why not translate SQL straight into one fixed execution strategy? Because the logical plan says what result is wanted, and *many* physical plans can produce it. Keeping them separate lets the optimiser:

- pick the best physical strategy for *today's* data and statistics (a plan that is fast on a small table may be terrible once the table has a hundred million rows);
- adapt as data grows or indexes are added, without the query being rewritten;
- apply general algebraic rewrites once, independently of execution de-

tails.

This is data independence in action, and it is why the same SQL keeps working as the system evolves. The cost is complexity: the optimiser is the hardest component to build, and when it guesses wrong (bad statistics, unusual data) the same query can suddenly run a hundred times slower. Chapter 5 is about making it guess well.

### Summary

Parse (grammar only) → bind (names, types, permissions, via the catalog) → rewrite (relational algebra, predicate pushdown) → optimise (choose access paths, join algorithms and order using statistics) → execute (iterator model: **open/next/close**, tuples pipelined up the tree). “Column does not exist” is caught at binding, not parsing.

## 4 Workloads and Storage Models

### 4.1 Two very different jobs: OLTP and OLAP

Databases serve two kinds of work that pull the design in opposite directions.

|                  | <b>OLTP</b> (online transaction processing) | <b>OLAP</b> (online analytical processing) |
|------------------|---------------------------------------------|--------------------------------------------|
| Purpose          | run the business                            | analyse the business                       |
| Typical query    | short, touches a few rows                   | long, scans millions of rows               |
| Writes           | many small inserts and updates              | bulk loads, few writes                     |
| Example          | “record this payment”                       | “revenue by region, last three years”      |
| Favoured storage | row store                                   | column store                               |

A payment system that processes ten thousand small transactions per second is OLTP. A dashboard that sums three years of sales is OLAP. The same data often lives in both: a row store for the day-to-day operations and a column store, refreshed nightly, for analytics. The *workload* decides the layout.

### 4.2 Row versus column storage

Take one logical table and lay it out on disk two ways.

| Logical table |      |        | Row store                                 | Column store                              |
|---------------|------|--------|-------------------------------------------|-------------------------------------------|
| id            | name | amount | 1, Sara, 40<br>2, Amir, 15<br>3, Lina, 90 | 1, 2, 3<br>Sara, Amir, Lina<br>40, 15, 90 |
| 1             | Sara | 40     |                                           |                                           |
| 2             | Amir | 15     |                                           |                                           |
| 3             | Lina | 90     |                                           |                                           |

In a **row store**, all fields of one record sit together, so fetching or updating a whole record touches one place. In a **column store**, all values of one column sit together, so scanning a single column reads only that column's data, and because every value in the run has the same type, it compresses extremely well.

#### Trade-offs: Which layout, when?

##### Choose a row store when

you read and write whole records;  
there are many small transactions;  
lookups are by key.

##### Choose a column store when

you scan a few columns over many rows;  
the work is aggregation and reporting;  
compression matters.

Rule of thumb: few rows, whole records  $\Rightarrow$  row store and OLTP. Many rows, few columns  $\Rightarrow$  column store and OLAP. Chapter 2 opens up the page layouts; Chapter 12 returns to this in the lakehouse setting.

#### Worked example: Pause and check: OLTP or OLAP, row or column?

1. An ATM withdrawal updating one account balance.
2. A dashboard showing total sales per region for the last three years.
3. Detecting fraud by scanning two years of transactions.
4. Changing a single user's email address.

1: OLTP, row store; one small record, read and written whole. 2: OLAP, column store; aggregate a couple of columns over many rows. 3: OLAP, column store; a large scan of a few columns. 4: OLTP, row store; a single-record point update.

#### Summary

OLTP is many short transactions on few rows; OLAP is few long scans over many rows. Row stores keep records together and suit OLTP; column stores keep columns together, compress well, and suit OLAP. The workload chooses the layout, and the same data may live in both.

## 5 SQL Refresher

We will use SQL in every lab, so here is the core, with the one thing most people never learn: the *logical order* in which the clauses are evaluated.

### 5.1 The core shape and its logical order

|                 |                  |                          |
|-----------------|------------------|--------------------------|
| <b>SELECT</b>   | column_list      | -- 5. what to return     |
| <b>FROM</b>     | table(s)         | -- 1. where to read from |
| <b>WHERE</b>    | row_condition    | -- 2. keep which rows    |
| <b>GROUP BY</b> | grouping_columns | -- 3. form groups        |
| <b>HAVING</b>   | group_condition  | -- 4. keep which groups  |

```
ORDER BY sort_columns;          -- 6. final order
```

The numbers give the logical evaluation order: FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY. Two consequences you will hit in practice. An alias defined in SELECT (AS total) cannot be used in WHERE, because WHERE runs first. And WHERE filters *rows* before grouping, while HAVING filters *groups* after; a condition on an aggregate such as COUNT(\*) > 100 can only go in HAVING.

**Key insight.** SQL is written SELECT-first but evaluated FROM-first. This is why you cannot use a SELECT alias in WHERE: WHERE runs before SELECT exists.

## 5.2 Joins

```
-- inner join: only matching pairs
SELECT c.name, o.amount
FROM   customers c JOIN orders o ON o.cust_id = c.id;

-- left join: keep all customers, even those with no orders (amount
              is NULL)
SELECT c.name, o.amount
FROM   customers c LEFT JOIN orders o ON o.cust_id = c.id;
```

On our tiny instance the inner join gives four rows (Sara 40, Lina 15, Sara 90, Amir 70). If we added a fourth customer with no orders, the inner join would omit them and the left join would include them with amount = NULL. *How* a join is computed (nested loop, sort-merge, hash) is Chapter 4; here we only say *what* to match.

## 5.3 Grouping and aggregation

```
SELECT   c.country, COUNT(*) AS n, AVG(o.amount) AS avg_amt
FROM     customers c JOIN orders o ON o.cust_id = c.id
GROUP BY c.country
HAVING   COUNT(*) > 1;
```

Aggregates: COUNT, SUM, AVG, MIN, MAX. Every column in the SELECT list must be either grouped by or inside an aggregate; otherwise the system has no single value to show for the group. On our instance: MA has three orders (40, 15, 90), FR has one; the HAVING keeps only MA, giving (MA, 3, 48.33).

## 5.4 Subqueries, DDL and DML

```
SELECT name FROM customers
WHERE id IN (SELECT cust_id FROM orders WHERE amount > 50);
```

The inner query yields the customer ids with a large order (1 and 2); the outer keeps those customers (Sara, Amir). The rewriter of Stage 3 will usually turn this into a join.

- **DDL** (data definition language) shapes the schema: CREATE, ALTER, DROP.

- **DML** (data manipulation language) moves data: `SELECT`, `INSERT`, `UPDATE`, `DELETE`.

## 6 Tricks, Techniques and Classic Problems

### Debugging: Tools for seeing inside the engine

- `EXPLAIN <query>`: the physical plan, with estimated rows and cost per operator. `EXPLAIN ANALYZE <query>`: runs it and adds actual rows and milliseconds. Read the tree bottom-up. A large gap between estimated and actual rows means stale statistics; run `ANALYZE <table>`.
- `\d customers` in `psql`: the catalog's view of a table (columns, types, indexes).
- `SELECT * FROM pg_stat_activity;` what every session is doing right now (subsystem 1 made visible).
- `SELECT * FROM pg_stats WHERE tablename = '...';` the statistics the optimiser uses.
- `SET enable_hashjoin = off;` then `EXPLAIN` again: force the optimiser to choose differently and compare costs.

### Technique: Reading an EXPLAIN plan

```
HashAggregate (cost=.. rows=2)
  Group Key: c.name
  -> Hash Join (cost=.. rows=3)
    Hash Cond: (o.cust_id = c.id)
    -> Seq Scan on orders o
    -> Hash
      -> Seq Scan on customers c
        Filter: (country = 'MA')
```

Indentation is the tree; the deepest lines run first. The side under **Hash** is the build side (customers, filtered), the other is the probe side (orders). That is exactly the plan we traced in Section 3.

### Warning / Gotcha: Common student mistakes

- **“The parser will catch a misspelled column.”** It will not. Parsing checks grammar only; the binder, consulting the catalog, catches it.
- **Using a `SELECT` alias in `WHERE`.** `WHERE total > 100` fails because `WHERE` is evaluated before `SELECT`. Repeat the expression, or use a subquery or `HAVING`.
- **Confusing `WHERE` and `HAVING`.** `WHERE` filters rows before grouping; `HAVING` filters groups after. Aggregates go in `HAVING`.
- **Assuming the optimiser executes the query as written.** It reorders joins and pushes filters down. The plan, not the SQL text, tells you what runs.
- **“The DBMS is one program.”** It is five subsystems; knowing which

one a problem belongs to tells you where to look (slow query: query processor or storage; lost update: transaction manager).

- **Treating the catalog as magic.** It is tables. Query it.

#### Worked example: Classic exam and interview problems

**P1. List the five stages a query passes through, in order.**

Parse → Bind → Rewrite (logical plan) → Optimise (physical plan) → Execute.

**P2. Name three checks that happen specifically during binding.**

(a) The tables named in FROM exist. (b) Every column exists on the table it is used with, and expressions type-check (SUM applied to a numeric column; = comparing compatible types). (c) The user has permission to read the tables.

**P3. At which stage is “column amuont does not exist” caught, and why not earlier?**

At binding. Parsing checks only grammar, and *amuont* is a perfectly valid identifier syntactically. Only when the binder looks it up in the catalog does it learn there is no such column.

**P4. What does the iterator model buy us?**

Uniform interfaces (*open/next/close*) so any operator can sit above any other, and pipelining: tuples flow one at a time, so intermediate results usually need not be materialised.

**P5. Which subsystem is responsible if two users’ concurrent updates lose one of the changes?**

The transaction manager (its lock manager). This is the lost-update anomaly, defined in Chapter 7.

**P6. Give one reason a query that was fast last month might be slow today with no change to the SQL.**

The data grew or its distribution changed, so the optimiser’s cost estimates changed and it picked a different physical plan; or statistics are stale and it picked a bad one. Run ANALYZE and compare EXPLAIN ANALYZE output.

**P7. Why is the filter on country placed below the join in the plan?**

Predicate pushdown: the filter mentions only customer columns, so it can run before the join, and doing so shrinks the join’s input. Fewer rows into the join means less work for every operator above.

#### Summary: Chapter 1 in six lines

1. A DBMS exists to give querying, concurrency, recovery, integrity and efficiency that flat files cannot.
2. You state *what* (SQL); the system chooses *how* (the plan). That gap is the course.
3. Five subsystems: connection manager, query processor, storage manager, transaction manager, catalog.
4. Five stages: parse, bind, rewrite, optimise, execute. Names and types are checked at binding, against the catalog.

5. Execution uses the iterator model: `open/next/close`, tuples pipelined up the plan tree.
6. OLTP favours row storage; OLAP favours column storage. Next: how rows become pages on disk.